

Project Phase 3: Advanced Structures and Graph Optimization

Released Week 10 • Due Week 14/15 • Weight: Part of the Semester Project (10%)

Objective

In **Phase 1**, you designed the object-oriented data model for the system. In Phase 2, you extended that same design using collections, sorting, searching, and efficiency analysis. In Phase 3, you will continue the **same Smart Logistics and Delivery Routing System** by adding advanced structures for indexing, hashing, and network routing.

This phase does not introduce a new project. You must extend and reuse the classes, relationships, and design decisions from Phases 1 and 2. Your work should remain consistent with the original project topic: packages, warehouses, customers, and delivery routes.

Phase 3 Snapshot

Area	Required Extension in Phase 3
Tree-based indexing	Use a BST or AVL tree to support ordered lookup of project objects by a unique key.
Hashing	Use a HashMap for direct access to another important set of project records.
Weighted graph	Model the delivery network as a weighted graph using cities, warehouses, or distribution locations as vertices.
Optimization algorithms	Apply Dijkstra for shortest path and Prim or Kruskal for a minimum spanning tree.
Design continuity	Preserve encapsulation, inheritance, interfaces, and the project entities created earlier.

1. System Evolution: From Collections to Advanced Structures

Using the same entities and relationships from your earlier phases (for example: DeliveryItem, Warehouse, Customer, DeliveryRoute, or your own equivalent classes), extend your system with the following required features.

A. Efficient Indexing with a Tree (Required)

- Implement one tree-based indexing structure to support fast ordered lookup of your main project objects.
- You must choose one: BST or AVLTree.
- Your tree must store and manage your own entity objects, not just primitive values.
- The tree must support at least: insertion, deletion, search by a unique key, and inorder traversal to show sorted order.
- Examples include package lookup by tracking number, shipment lookup by ID, warehouse lookup by code, or route lookup by route ID.
- You may adapt the BST or AVL code from class, or design a consistent variation that fits your project.
- If you use an AVL tree, explain briefly why guaranteed balance is useful for your selected data.

B. Hashing for Fast Record Access (Required)

- Use a `HashMap<K, V>` to support direct access to another important part of your system in expected $O(1)$ time.
- Examples include customer lookup by customer ID, package lookup by tracking number, or warehouse lookup by name or code.
- In your documentation, justify your key and value types and explain why hashing is useful in your design.
- Your `HashMap` demonstration must show insertion, retrieval, and deletion or update.

C. Weighted Graph Representation (Required)

- Model part of your system as a weighted graph.
- Vertices may represent cities, warehouses, delivery hubs, or another meaningful logistics location in your project.
- Edges must represent a meaningful delivery relationship such as road distance, travel time, or shipping cost.
- Your graph must have at least 6 vertices and weighted edges.
- You may represent the graph using either an adjacency list or an adjacency matrix, but your code and documentation must make the representation clear.

2. Algorithmic Features: Routing and Network Optimization

A. Shortest Path (Required)

- Implement Dijkstra's algorithm on your weighted graph.
- Your program must demonstrate the shortest path from one source vertex to one destination vertex.
- Your output must include both the total cost and the actual path in readable order.
- Example output: Houston -> Dallas -> Austin (total distance or cost shown clearly).

B. Minimum Spanning Tree (Required)

- Implement one of the following: Prim's algorithm or Kruskal's algorithm.
- Your program must demonstrate the edges selected in the MST and the total weight of the MST.
- In your documentation, explain briefly what the MST means in your project context, such as the minimum infrastructure cost to connect all warehouses or the minimum total route cost to maintain network coverage.

3. Required Design Continuity

- Phase 3 must continue to reuse and preserve ideas from the earlier phases. This is an extension of the same system, not a redesign from scratch.
- Your code must still demonstrate encapsulation, inheritance and polymorphism, interfaces where appropriate, generics where appropriate, and reuse of your Phase 1 and Phase 2 entities and managers.

4. Implementation Requirements (Must Be Visible in Code)

- Your Phase 3 code must clearly show all of the following: one BST or AVLTree, one `HashMap`, one weighted graph representation, Dijkstra's algorithm, Prim's or Kruskal's algorithm, and a driver/test program that demonstrates all required features.
- In your documentation, also justify your design choices briefly: why you selected BST or AVL, what role the `HashMap` plays, why graph modeling is appropriate, and how the routing algorithms improve your system.

5. Testing and Verification

- Update your existing test/driver class or create a new driver class for Phase 3.
- Your test program must demonstrate all required tree, hashing, and graph operations clearly using sample data from your project.

Required demonstrations in the test program

Area	What your test/driver must show
Tree demonstration	Insert several objects into the tree; search for an existing key; search for a missing key; delete one object; print the tree contents in sorted order.
HashMap demonstration	Add records into the map; retrieve an existing record; handle a missing key properly; update or remove a record.
Graph demonstration	Print or display the graph structure; run Dijkstra from a source to a destination; print the shortest path and total cost; run Prim or Kruskal; print the MST edges and total weight.
Project continuity	Use the same project entities and context from the earlier phases rather than unrelated stand-alone examples.

6. Efficiency Analysis (Required)

- Provide a brief Big-O discussion for tree insertion, search, and deletion; HashMap access; Dijkstra's algorithm; and Prim's or Kruskal's algorithm.
- You do not need a long report, but you must explain why these structures and algorithms are efficient for your project.

7. Submission Package (Single ZIP File)

1. Phase 3 Documentation (PDF)

Include: project title; a short explanation of how your system evolved from Phase 2; tree structure choice and justification; HashMap usage and justification; graph design description; shortest path explanation; MST explanation; Big-O discussion; and screenshots or sample output if helpful.

2. Source Code

Include all updated classes from Phases 1 and 2, tree classes or adapted tree classes, graph-related classes, manager/controller classes, and code for Dijkstra and Prim or Kruskal.

3. Test Program

Include a driver class that loads or generates data, demonstrates tree operations, demonstrates HashMap operations, demonstrates shortest path, demonstrates MST, and prints clear results to the console.

8. Submission Checklist

- ☐ One BST or AVLTree integrated into the project
- ☐ One HashMap integrated into the project
- ☐ One weighted graph with at least 6 vertices
- ☐ Dijkstra's algorithm
- ☐ Prim's or Kruskal's algorithm
- ☐ Clear testing output
- ☐ Documentation PDF
- ☐ Reuse of earlier project classes and design choices
- ☐ Single ZIP file uploaded to Canvas by the deadline

9. Notes to Students

- You may use ideas from class examples, but your final code must fit your own Smart Logistics and Delivery Routing System.
- Your tree and graph must work with your project entities, not as isolated unrelated examples.
- Keep the design simple and consistent. Correctness, clarity, and integration matter more than unnecessary complexity.
- You may discuss general ideas with peers, but all code must be your own original work. Use of AI tools for brainstorming is permitted only with clear attribution, consistent with the course project policies.

Reminder: Phase 3 extends the same project from Phases 1 and 2. The strongest submissions will show continuity, clear testing, and meaningful integration of trees, hashing, and graph algorithms within the logistics system.